

Proper Whole Summary: Agentic Graph RAG Assistant

1. Overview

The **Agentic Graph RAG Assistant** is a fully functional, intelligent information retrieval system that operates entirely locally. It combines traditional Retrieval-Augmented Generation (RAG) with a Knowledge Graph to offer rich context, powered by an agentic workflow that reflects on its own results before answering. The entire architecture relies on local LLM inference (via Ollama) to ensure privacy and efficiency.

2. Core Concepts Used

A. Agentic Workflow (LangGraph)

Unlike a linear RAG pipeline that always answers based on the first retrieval, this system employs an **Agentic Workflow** built using `LangGraph`. It treats the QA process as a state machine with the following nodes:

- **Retrieve**: Fetches context from both Vector database and Knowledge Graph.
- **Grade Documents**: Uses a local LLM to evaluate if the retrieved documents actually answer the question or are relevant.
- **Generate**: If relevant context is found, it generates a well-reasoned answer.
- **Rewrite**: If the context was deemed *irrelevant*, the agent re-writes the user's query to perform a better, optimized search.
- **Route Decision**: A conditional edge automatically decides whether to proceed to "Generate" or "Rewrite" based on the grading step.

B. Hybrid Retrieval (Vector + Graph RAG)

The system leverages a hybrid approach to retrieval:

1. **Vector Retrieval**: Semantic similarity search using **Chroma DB**. It matches the user's question against chunked document embeddings to find the top 5 most similar passages.
2. **Graph Retrieval**: Uses **NetworkX** to maintain an entity relationship graph. During retrieval, the LLM extracts 1-3 key entity nouns from the query. The system then searches the graph for these entities and pulls adjacent nodes and their relation edges (e.g., `Node A --is related to--> Node B`). This provides structural context that standard vectors might miss.

C. Local AI Stack

- **LLM Inference**: Handled by **Ollama**, utilizing explicit local models (default: `llama3`). This guarantees zero API costs and full data privacy.
- **Embeddings**: Utilizes the high-efficiency HuggingFace model `all-MiniLM-L6-v2` to convert text chunks into vector representations locally.

D. Data Ingestion Pipeline

- **Loaders**: Supports documents like PDFs (via `PyPDFLoader`) and text files (via `TextLoader`).
- **Chunking**: Uses `RecursiveCharacterTextSplitter` to break documents into 1000-character chunks with a 200-character overlap, preserving context across boundaries.
- **Graph Extraction**: During ingestion, an `LLMGraphTransformer` parses the text chunks to explicitly identify nodes (entities) and edges (relationships) to populate the `.gml` Knowledge Graph.

3. Working Architecture

1. **Ingestion Phase:** A user uploads a file through the FastAPI `/api/upload` endpoint. The backend triggers the ingestion pipe: files are chunked, embedded into Chroma DB, and their structural entities are evaluated and added to the NetworkX Knowledge Graph.
2. **Querying Phase:** When a prompt hits `/api/chat`, the LangGraph agent receives the query. It retrieves hybrid context (Vectors + Graph), grades it, and ultimately generates a final response or rewrites the prompt to try again up to the defined recursion limit.
3. **Frontend-Backend Integration:** The backend employs **FastAPI** to securely serve these agentic capabilities while mounting the static frontend files, providing a seamless modern web UI.